◆ TECHNICAL RESEARCH REPORT

AI-Driven SOC Triage & Threat Hunting Pipeline

Automating Tier-1 Triage with Llama 3 + Wazuh SIEM

Architected by **Aditya Mohanty**

linkedin.com/in/adityamohanty-security

92%	91%	90%	~4M
Accuracy	Precision	Recall	Unfilled Roles

01 | Problem Statement

The global cybersecurity landscape is destabilized by a critical workforce deficit, with estimates ranging from **1.8 million to 4 million unfilled positions**. This shortage directly exacerbates *Alert Fatigue* — a phenomenon where **31.9% of security analysts admit to ignoring alerts** due to overwhelming volume. Conventional SOCs are burdened by high false-positive rates from rule-based systems, leading to analyst burnout and increased Mean Time to Respond (MTTR).

Mission: Architect an automated Tier-1 triage and proactive threat hunting pipeline by integrating Meta's **Llama 3 LLM** with the **Wazuh SIEM** platform — automating event classification and delivering natural-language insights to empower human analysts.

02 | Core SIEM Architecture: Wazuh

Wazuh Manager	Central engine at /var/ossec/. Normalizes logs, applies rule engine, manages agent communication.
Wazuh Indexer	High-performance search & analytics engine storing alerts and metadata for sub-second retrieval.
Wazuh Dashboard	Visualization layer: alert monitoring, configuration management, and the integrated AI analyst interface.

Log Collection & Normalisation

Wazuh ingests logs via **decoders** (parsing raw text to structured JSON) and **rules** (triggering alerts on attack patterns). The **archives.json** feature stores every ingested log regardless of whether it triggered a rule — providing the raw data pool for the AI to identify stealthy, sub-threshold anomalies.

03 | Python Automation Engine

3.1 threat_hunter.py — Core Logic

- **Distributed Ingestion** — Paramiko library ingests logs from remote Wazuh managers via SSH/SFTP.
- **Vectorization** — Logs loaded into a FAISS (Facebook AI Similarity Search) vector store for semantic retrieval.
- **Embeddings** — all-MiniLM-L6-v2 via HuggingFaceEmbeddings generates numerical log representations.
- **Hardware** — Minimum 16 GB RAM and 4 CPUs to co-host Llama 3 (Ollama) alongside Wazuh Manager.

3.2 Technology Stack

Library / Component	Function within Pipeline
`FastAPI`	Web framework for the AI chatbot, serving on Port 8000
`LangChain`	Orchestrates RAG (Retrieval-Augmented Generation) and conversation chains
`Ollama`	Local inference engine for Llama 3 — ensures data privacy
`Sentence-Transformers`	Powers all-MiniLM-L6-v2 for log embeddings
`Paramiko`	Facilitates remote log collection from Wazuh managers

`Uvicorn`	ASGI server for high-performance FastAPI execution

3.3 Natural Language Interface & MCP Integration

A web-based chatbot (FastAPI) exposes the AI to analysts. Advanced integration via the **Model Context Protocol (MCP)** enables execution of security tools via natural language:

- firewall-block — immediate automated remediation
- analyze-cti-trends — external threat intelligence correlation
- automated-incident-creation — ticketing workflow integration

04 | AI Analyst Inference: Llama 3

4.1 Model Selection & Local Deployment

Llama 3 (8B and 70B variants) is deployed locally via **Ollama**. Local deployment is non-negotiable for SOC environments — preventing sensitive internal logs from being exposed to third-party cloud providers.

4.2 Alternative Inference Providers

Provider	Use Case
Groq / Cerebras	Low-latency inference via LPUs for near-instantaneous real-time triage
OpenRouter	Provider redundancy and model variety through a unified API gateway

4.3 Fine-tuning: wazuh-model

We utilized **wazuh-model**, a specialized fine-tune of Llama 3.1 8B trained exclusively on Wazuh alerts of Level 3 and above. For consumer-grade hardware (T4 GPUs), the model uses **4-bit quantization** via the Unsloth framework — balancing high-fidelity classification with minimal VRAM utilization.

05 | AI-Enhanced SOC Capabilities

Automated Alert Triage

Classifies alerts as 'interesting' (high risk) or 'not-interesting' (benign). Automatically performs Base64 decoding of PowerShell strings, revealing hidden scripts that traditional signatures often miss.

Contextualisation & Threat Hunting

Analyses process trees to identify sophisticated reconnaissance — e.g. cmd.exe spawning cmdkey.exe for credential harvesting, or sequences of net user/net localgroup commands flagging unauthorized enumeration.

Rule Generation

When benign activity repeatedly triggers alerts, the AI generates Wazuh exclusion rules in correct XML syntax for /var/ossec/etc/rules/ — copy-paste validated exclusions that significantly reduce future alert noise.

06 | DevOps & Ruleset-as-Code (RaC) Pipeline

All detection logic is managed with engineering discipline via a CI/CD workflow:

1. Local Dev	Engineers write rules in VS Code / IDE

2. Push to Dev	Rules pushed to remote dev branch
3. Pull Request	check_rule_ids.py runs — prevents ID conflicts
4. Merge	GitHub Actions triggered on merge to main
5. Deploy	git pull --rebase on Wazuh Manager + chown/chmod permissions set
6. Restart	wazuh-manager service restarted to apply new detection logic

07 | Adversary Emulation & Testing Results

Validation was conducted using the **CALDERA emulation framework** to simulate realistic threat actor TTPs.

! Brute-Force Simulation

- SSH brute-force targeting manager public IP
- Users ubuntu and admin tested with common passwords
- LLM identified failed-login pattern and pinpointed source IP

* Data Exfiltration Simulation

- PowerShell Invoke-WebRequest to remote Netcat listener
- AI identified responsible user, binary, and destination IP
- Event correctly classified as high-risk

Performance Metrics — CALDERA Evaluation Framework

92%	91%	90%
Accuracy	Precision	Recall

08 | Challenges & Limitations

! Hallucinations

LLMs may generate inaccurate security summaries. Human-in-the-loop verification remains critical for final incident confirmation.

! Integration Complexity

Bridging Wazuh, FAISS, and LLMs requires constant maintenance of Python automation scripts and API versions.

! Privacy Risks

While local models mitigate data leakage, any fallback to cloud providers (e.g. OpenRouter) must be strictly governed by data privacy policies.

09 | Conclusion & Future Roadmap

This AI-driven pipeline effectively mitigates the global skill shortage impact by automating Tier-1 tasks and reducing MTTR through natural language context. Treating the detection ruleset as code ensures a scalable, stable, and auditable security environment.

-> Prompt Optimisation	Specialised system prompts to further decrease hallucination rates.
-> Dataset Expansion	Training wazuh-model on larger multi-platform (Linux/macOS) logs to improve global recall.
-> Active Remediation	Integrating AI decision-making with Wazuh Active Response scripts for fully automated threat containment.

Aditya Mohanty · linkedin.com/in/adityamohanty-security